AI Agent-Powered Music Video Production: A ComfyUI Implementation Guide

Report Date: 2026-03-22

This document provides a comprehensive implementation guide for designing, building, and deploying an advanced AI agent system that orchestrates ComfyUI to automate the creation of music videos. The guide covers system architecture, technology recommendations, hardware management, and a detailed step-by-step pipeline from initial concept to final video output.

1. System Architecture

The proposed system is a modular, multi-component architecture designed for robust and scalable music video generation. It decouples the user interface, intelligent orchestration, and generative processing into distinct services that communicate via APIs. This design ensures flexibility, allowing individual components to be updated or scaled independently.

The core components of the architecture are:

- **Web Interface (Frontend):** A user-facing application that serves as the primary interaction point. Users submit their music track, lyrics, and high-level creative direction (e.g., style, mood, themes) through this interface. It is responsible for initiating the generation process and displaying the final output.
- **Agent Orchestrator (Backend):** This is the central intelligence of the system. Built upon a sophisticated AI agent framework, it receives user requests and autonomously plans and executes the entire music video creation pipeline. It interprets user input, directs the Large Language Model (LLM) for creative tasks, manages tools, and communicates with the ComfyUI engine.
- **LLM Inference Server:** A dedicated, high-performance server hosting a large-scale language model. This server exposes an API that the Agent Orchestrator calls to perform complex reasoning tasks, such as analyzing lyrics, developing a narrative scene plan, and generating detailed visual prompts for ComfyUI.
- **ComfyUI Server:** The powerful node-based generative engine responsible for creating all visual assets. It runs as a persistent service, exposing an HTTP and WebSocket API. The Agent Orchestrator sends it dynamically constructed workflow payloads to generate image sequences or video clips for each scene.
- **Tool Library:** A collection of specialized Python functions and scripts that the Agent Orchestrator can invoke as “tools.” These tools provide concrete capabilities, such as analyzing audio files to detect beats, interacting with the ComfyUI API, managing files on the system, and assembling final video files.
- **Shared Storage:** A centralized storage solution (e.g., a network-attached drive or a cloud bucket) accessible by all components. This repository holds essential assets, including the foundational AI models (checkpoints, LoRAs), user-uploaded files, intermediate generated clips, and the final video outputs.

The operational flow begins at the Web Interface, where a user's request is sent to the Agent Orchestrator. The agent then follows a multi-step plan, first calling the Tool Library to analyze the audio. It then queries the LLM Inference Server to create a scene plan and prompts. Subsequently, it uses another tool to send these prompts within a workflow to the ComfyUI Server for generation. Finally, it assembles the resulting clips into a complete video, which is then made available to the user through the Web Interface.

2. Technology Stack Recommendations

The selection of an appropriate technology stack is critical for the system's performance, stability, and scalability. The following recommendations are based on proven open-source technologies well-suited for this AI-driven creative workflow.

- **Agent Orchestrator:** Python is the language of choice due to its extensive AI/ML ecosystem. The **LangGraph** framework is highly recommended for building the agent. Its ability to create stateful, cyclical graphs is ideal for managing the complex, multi-step, and potentially iterative process of video creation, including support for human-in-the-loop validation. **AutoGen** is a strong alternative, particularly for scenarios envisioning collaboration between multiple specialized agents.
- **LLM Inference Server:** The **vLLM** framework is the top recommendation for serving the LLM. It is engineered for high-throughput and efficient memory management on NVIDIA GPUs, supporting tensor parallelism to run a single large model across multiple GPUs. It provides an OpenAI-compatible API endpoint, simplifying integration with agent frameworks.
- **Large Language Model (LLM):** For creative planning and detailed prompt generation, a powerful instruction-tuned model is required. The **Llama 3 70B** or **Qwen2 72B** models are excellent choices. To fit within consumer-grade hardware, they should be used in a 4-bit quantized format (e.g., AWQ or GGUF compatible with the chosen server). These models demonstrate strong reasoning and function-calling capabilities.
- **Generative Engine:** The core of the visual creation process is **ComfyUI**. It should be run as a standard server instance to expose its native API.
- **Audio Analysis:** The **Librosa** and **Essentia** Python libraries are recommended for the audio analysis tool. Librosa is excellent for fundamental tasks like beat and tempo detection, while Essentia offers more advanced capabilities like drop detection, which can be used to trigger more dramatic visual changes.
- **ComfyUI Interaction:** The agent's tool for communicating with ComfyUI can be built using standard Python libraries: **`requests`** for HTTP calls (e.g., to `/prompt` and `/history`) and **`websockets`** for real-time progress monitoring via the `/ws` endpoint.
- **Web Interface:** While the specific choice is flexible, modern JavaScript frameworks like **React**, **Vue.js**, or **Svelte** are suitable for building a responsive and interactive frontend.
- **Containerization: Docker** and **Docker Compose** are strongly recommended for packaging and deploying the various services (Agent Orchestrator, vLLM Server, ComfyUI Server). Containerization simplifies dependency management, ensures consistency across environments, and facilitates deployment.

3. Hardware Resource Management

The simultaneous operation of a large language model and a diffusion-based generative model is resource-intensive, demanding careful management of GPU memory (VRAM) and processing power. The recommended hardware configuration is a system with two NVIDIA RTX 3090 GPUs, providing a total of 48GB of VRAM, and at least 128GB of system RAM.

A critical performance enhancement for this dual-GPU setup is the use of an **NVLink bridge**. This high-speed interconnect allows the two GPUs to share memory far more efficiently than over the standard PCIe bus, potentially boosting LLM inference performance by up to 50%.

The most stable and recommended strategy for managing GPU resources is **dedicated allocation**. This involves assigning each major workload to its own GPU to prevent VRAM contention. This is achieved by setting the `CUDA_VISIBLE_DEVICES` environment variable when launching each service:

- **LLM Inference Server (vLLM):** This is the most VRAM-hungry component. A 4-bit quantized 70B-parameter model requires approximately 42-45 GB of VRAM. Using vLLM's tensor parallelism, the model can be split across both GPUs. However, a simpler and highly effective approach is to dedicate one GPU entirely to ComfyUI and run a slightly smaller but still powerful model on the other. For instance, a Mixtral 8x7B model requires only 26-28GB of VRAM, fitting comfortably on one RTX 3090 and leaving the other 24GB GPU completely free for ComfyUI. If running the 70B model is a priority, it must be split across both GPUs.
- **ComfyUI Server:** This service should be assigned to the other GPU. A 24GB RTX 3090 provides ample VRAM for running complex ComfyUI workflows, including those using high-resolution models, ControlNets, and video generation nodes like AnimateDiff.

An alternative, more advanced strategy involves dynamic offloading using the **`ComfyUI-MultiGPU`** custom node. This extension allows for fine-grained control over where different parts of a ComfyUI model (e.g., UNet, CLIP) are loaded. It can offload components to a secondary GPU or even to system RAM. This could enable running a smaller LLM and ComfyUI concurrently on the same primary GPU by freeing up just enough VRAM, though the dedicated GPU approach remains preferable for stability and performance.

4. Step-by-Step Setup Instructions

This section outlines the necessary steps to prepare the environment for the AI agent system.

1. **System Prerequisites:** Ensure your system has the latest NVIDIA drivers, the appropriate CUDA Toolkit version compatible with the chosen frameworks (vLLM, PyTorch), Python 3.10 or newer, and Git.
2. **ComfyUI Installation:**
    - Clone the ComfyUI repository: `git clone https://github.com/comfyanonymous/ComfyUI.git`.
    - Navigate into the directory and install the required Python packages: `pip install -r requirements.txt`.
    - It is highly recommended to install the **ComfyUI Manager** to simplify the installation of custom nodes and models.

3. **Install Key Custom Nodes:** Using the ComfyUI Manager, search for and install the following custom nodes, which will be essential for agent-driven workflows:
    - Nodes for video generation (e.g., **AnimateDiff Evolved**).
    - Nodes for advanced batch processing (e.g., **WAS Node Suite**).
    - Nodes for LLM integration if direct communication from within ComfyUI is desired (e.g., **ComfyUI-OpenAI-Compat-LLM-Node**).
4. **LLM Inference Server Setup (vLLM):**
    - Install vLLM: `pip install vllm`.
    - Download a 4-bit quantized model from Hugging Face (e.g., a model in the AWQ format).
    - Launch the vLLM server. To run a model across two GPUs, use the `tensor-parallel-size` flag. For example:
      `python -m vllm.entrypoints.openai.api_server --model <huggingface-repo-id> --tensor-parallel-size 2 --dtype auto`
    - This will start an API server at `http://localhost:8000`, which is compatible with the OpenAI API specification.
5. **Agent Environment Setup:**
    - Create a new Python virtual environment for the orchestrator to isolate its dependencies.
    - Install the chosen agent framework: `pip install langgraph`.
    - Install necessary libraries for the toolset: `pip install requests websockets librosa essentia`.
6. **Launch Services with GPU Allocation:**
    - Open two separate terminal sessions.
    - In the first terminal, launch the vLLM server, assigning it to GPU 0:
      `CUDA_VISIBLE_DEVICES=0 python -m vllm.entrypoints.openai.api_server ...`
    - In the second terminal, launch the ComfyUI server, assigning it to GPU 1:
      `CUDA_VISIBLE_DEVICES=1 python main.py`
    - The Agent Orchestrator can then be run on the CPU or share a GPU if its own memory footprint is minimal.

5. Agent Implementation

The Agent Orchestrator is the system's brain. Using the **LangGraph** framework is recommended for its explicit state management and control flow, which are perfect for the structured, multi-step task of video generation.

The agent is designed as a stateful graph where each node represents a step in the process and edges define the transitions.

Agent State

The state is a Python dictionary that is passed between nodes, accumulating information as the process unfolds.

```python
from typing import TypedDict, List, Dict

class AgentState(TypedDict):
    music_path: str
    lyrics: str
    style_prompt: str
    audio_analysis: Dict  # To store BPM, beat timestamps, etc.
    scene_plan: List[Dict] # A list of scenes with descriptions
    generated_prompts: List[str]
    generated_clips: List[str] # Paths to intermediate video clips
    final_video_path: str
```

Agent Tools

The agent's capabilities are defined by a set of tools it can call. These are Python functions decorated to be accessible by the agent.

- **`AudioAnalysisTool`:** This tool takes a file path to the music track, uses Librosa to extract the BPM and a list of beat timestamps, and returns this data in a structured JSON format.
- **`CreativePlannerTool`:** This tool communicates with the LLM server. It takes the lyrics, audio analysis data, and user's style prompt, and asks the LLM to generate a narrative arc and a structured scene plan. The LLM's ability to perform function calling is leveraged here to ensure it returns a well-formed JSON list of scenes.
- **`ComfyUIOrchestratorTool`:** This is the most critical tool. It encapsulates all interaction with the ComfyUI API. It has methods to:
    1. Load a template workflow JSON file.
    2. Programmatically modify the JSON to insert prompts, seeds, or file paths.
    3. Send the modified workflow to the `/prompt` endpoint to queue the job.
    4. Connect to the `/ws` WebSocket to monitor the job's real-time progress.
    5. Fetch the output file information from the `/history/{prompt_id}` endpoint upon completion.
    6. Download the generated clip using the `/view` endpoint.
- **`VideoAssemblyTool`:** A simple tool that uses a library like `moviepy` or a command-line call to `ffmpeg` to concatenate the individual scene clips into a single final video file.

Agent Graph

In LangGraph, these tools and state are connected in a graph:

1. **`analyze_audio` Node:** The entry point. It calls the `AudioAnalysisTool` and updates the state with the audio data.
2. **`plan_scenes` Node:** Calls the `CreativePlannerTool` (LLM) to generate the scene plan and prompts, updating the state.
3. **`human_in_the_loop` Node (Optional but Recommended):** A conditional node that pauses the graph and waits for user approval of the scene plan and prompts before proceeding to the computationally expensive generation step.
4. **`generate_scenes` Node:** A loop that iterates through the scene plan. In each iteration, it calls the `ComfyUIOrchestratorTool` to generate the video clip for that scene and adds the file path to the `generated_clips` list in the state.
5. **`compile_video` Node:** After the loop completes, this node calls the `VideoAssemblyTool` to create the final video.

6. `END`: The graph execution finishes.

6. Web Interface Design

The web interface should be clean, intuitive, and focused on guiding the user through the creative process. A conceptual design includes three main sections:

- **Input & Configuration:** This is the starting point.
  - A large file upload component for the music track (.mp3, .wav).
  - A text area for the user to paste lyrics. An optional feature could be an AI-powered transcription service.
  - A primary text field for the **“Overall Style & Vision,”** where the user describes the desired aesthetic (e.g., “Ghibli-style animation,” “dark, cyberpunk noir,” “surreal watercolor painting”).
  - A separate text field for **“Negative Prompts”** to specify elements to avoid.
  - A prominent “Generate Music Video” button to submit the request.
- **Process Visualization & Review:** This section provides transparency into the agent’s work.
  - A status indicator showing the current stage of the pipeline (e.g., “Analyzing Audio,” “Planning Scenes,” “Generating Scene 3 of 12”).
  - A “thought process” log where the agent outputs key decisions and actions in natural language.
  - If the human-in-the-loop step is enabled, this section will display the LLM-generated scene plan and prompts in an editable format, with “Approve” and “Reject” buttons.
- **Output & Delivery:** The final result is presented here.
  - An embedded video player displaying the final, fully rendered music video.
  - A “Download Video” button.
  - A gallery of keyframes or short clips from each generated scene.

7. Workflow Management

The system’s ability to generate varied and high-quality visuals hinges on its management of ComfyUI workflows. A template-based approach is the most effective and maintainable strategy.

- **Workflow Templates:** Create a library of pre-built ComfyUI workflows saved in the “API format” JSON. This format is optimized for programmatic use as it is more compact and self-contained than the standard UI save format. Different templates can be designed for specific styles or techniques (e.g., an AnimateDiff workflow, a frame-by-frame img2img workflow, a workflow with specific ControlNets).
- **Dynamic Modification:** The `ComfyUIOrchestratorTool` is responsible for the dynamic manipulation of these templates. Before each generation task, the agent will instruct the tool to:
  1. Load the appropriate template JSON into a Python dictionary.
  2. Identify the target nodes for modification, typically by their `class_type` or a unique title assigned in the ComfyUI interface.

3. Update the values of key widgets within these nodes. The most common modifications include:
   - **`CLIPTextEncode`:** Updating the `text` input with the LLM-generated prompt for the current scene.
   - **`KSampler`:** Changing the `seed` to ensure variation between scenes, and potentially modulating the `denoise` value to control the degree of change from one frame to the next.
   - **`LoadImage`:** Setting the `image` filename to provide a consistent character or style reference image.
4. Serialize the modified dictionary back into a JSON string to be sent in the API request.

This method allows the agent to reuse a single, robust workflow structure while dynamically injecting the creative content for each part of the music video.

8. Music Video Pipeline

The end-to-end pipeline, orchestrated by the agent, transforms a song into a synchronized music video through a sequence of automated steps.

1. **Initiation:** The user submits a song, lyrics, and style guidance via the web UI. The request is forwarded to the Agent Orchestrator, initializing the agent's state.
2. **Audio Analysis:** The agent's first action is to invoke the `AudioAnalysisTool`. The tool processes the audio file and returns a data structure containing the song's BPM and a precise list of beat timestamps. This rhythmic data is added to the agent's state.
3. **Creative Planning (LLM):** The agent formulates a request for the LLM, providing the lyrics, the user's style prompt, and the extracted audio data. It instructs the LLM to act as a creative director, breaking the song into thematic scenes and generating a short, evocative description for each. The output is a structured list of scenes.
4. **Prompt Generation (LLM):** The agent iterates through the scene plan. For each scene, it makes another call to the LLM, asking it to expand the brief description into a detailed, high-quality visual prompt suitable for a diffusion model. This prompt incorporates the scene's narrative elements with the user's overall stylistic keywords.
5. **Human Review (Optional):** The agent presents the full scene plan and the detailed prompts to the user for review. The process pauses until the user approves the creative direction, preventing wasted computation on an undesirable concept.
6. **Iterative Scene Generation:** The agent enters a loop, processing one scene at a time. In each iteration:
   - It selects an appropriate ComfyUI workflow template.
   - It uses the `ComfyUIOrchestratorTool` to inject the scene's prompt, a new seed, and any other dynamic parameters into the workflow JSON.
   - It sends the job to the ComfyUI server via the `/prompt` API endpoint.
   - It monitors the WebSocket for progress and completion signals.
   - Upon completion, it retrieves the path to the generated video clip and stores it.
7. **Final Assembly:** Once all scene clips have been generated, the agent calls the `VideoAssemblyTool`. This tool stitches the clips together in the correct order, creating the final, full-length music video.

8. **Delivery:** The path to the final video is added to the agent's state, and the result is made available to the user for viewing and download in the web interface.

9. Code Examples

The following snippets illustrate key parts of the implementation.

LangGraph Agent State

```
# In your agent's main Python file
from typing import TypedDict, List, Dict, Annotated
import operator

class AgentState(TypedDict):
    """Represents the state of our music video generation agent."""
    music_path: str
    lyrics: str
    style_prompt: str
    # The 'messages' field is used for LLM interactions
    messages: Annotated[list, operator.add]
    audio_analysis: Dict
    scene_plan: List[Dict]
    current_scene_index: int
    generated_clips: List[str]
    final_video_path: str
```

ComfyUI Orchestrator Tool Snippet

```
# In your tools/comfyui_tool.py file
import json
import requests
import uuid

COMFYUI_URL = "http://127.0.0.1:8188" # Default ComfyUI address

def queue_comfyui_prompt(prompt_workflow: dict) -> str:
    """Queues a workflow and returns the prompt ID."""
    client_id = str(uuid.uuid4())
    payload = {"prompt": prompt_workflow, "client_id": client_id}
    headers = {'Content-Type': 'application/json'}

    response = requests.post(f"{COMFYUI_URL}/prompt", data=json.dumps(payload), head-
ers=headers)
    response.raise_for_status()
    return response.json()['prompt_id']

def modify_workflow_prompt(workflow: dict, node_id: str, new_prompt: str) -> dict:
    """Finds a CLIPTextEncode node and updates its text."""
    # This is a simplified example; a robust version would handle different node types
    node = workflow.get(node_id)
    if node and node.get("class_type") == "CLIPTextEncode":
        node["inputs"]["text"] = new_prompt
    return workflow
```

vLLM Server Launch Command

```
# Launch vLLM with an OpenAI-compatible endpoint, running a model across 2 GPUs
CUDA_VISIBLE_DEVICES=0,1 python -m vllm.entrypoints.openai.api_server \
    --model "TheBloke/Llama-3-70B-Instruct-AWQ" \
    --quantization awq \
    --tensor-parallel-size 2 \
    --max-model-len 4096
```

Configuring a Local LLM in LangGraph

```
# In your agent's graph definition file
from langchain_openai import ChatOpenAI

# Point the LLM client to the local vLLM server
local_llm = ChatOpenAI(
    model="TheBloke/Llama-3-70B-Instruct-AWQ", # Model name used by vLLM
    openai_api_key="not-needed",
    openai_api_base="http://localhost:8000/v1"
)

# This 'local_llm' object can now be used in the agent's nodes
```

10. Best Practices

To ensure the system is robust, maintainable, and produces high-quality results, adhere to the following best practices.

- **Modularity and Abstraction:** Design tools to be self-contained and single-purpose. The agent should not know the implementation details of how ComfyUI works; it should only know how to use the `ComfyUIOrchestratorTool`. This makes the system easier to debug and upgrade.
- **Resource Isolation:** For production or heavy use, strictly enforce the dedicated GPU strategy using `CUDA_VISIBLE_DEVICES`. This prevents out-of-memory errors and unpredictable performance drops caused by resource contention.
- **Robust Error Handling:** All API calls (to the LLM server and ComfyUI) should be wrapped in try-except blocks with logging and retry mechanisms. Network failures or temporary server unavailability should not cause the entire multi-hour generation process to fail.
- **Stateful and Idempotent Operations:** Design the agent's steps to be idempotent where possible. If the process fails while generating scene 5, it should be possible to resume from that point using the saved state, rather than starting over from scratch.
- **Prioritize Human-in-the-Loop:** The creative planning and prompt generation steps are fast and cheap, while the video generation is slow and expensive. Always include a human review checkpoint before generation begins to avoid wasting significant time and energy on a flawed concept.
- **Workflow Version Control:** Treat your ComfyUI workflow JSON templates as code. Store them in a Git repository to track changes, experiment with new versions, and revert to previous working versions if needed.

- **Maintain Visual Consistency:** To avoid a disjointed final video, employ techniques for stylistic cohesion. Use the same reference images (for characters or styles via IPAdapters) and consistent stylistic keywords in the prompts for all scenes.

11. Deployment Considerations

Moving the system from a local development environment to a production-ready deployment requires additional planning for scalability, reliability, and accessibility.

- **Containerization:** Package each service (Web UI, Agent Orchestrator, vLLM Server, ComfyUI Server) into its own Docker container. Use Docker Compose to define and manage the multi-container application. This standardizes the deployment and simplifies scaling.
- **Scalability:** The initial bottleneck will likely be the ComfyUI generation step. For horizontal scaling, consider replacing the standard ComfyUI server with a solution like the `comfyui-api` server by SaladTechnologies. This stateless Node.js wrapper is designed to run multiple ComfyUI instances behind a load balancer, allowing you to process multiple generation jobs in parallel. It also adds production-grade features like webhook support for asynchronous job completion and integration with cloud storage backends like S3.
- **Asynchronous Processing:** For a production web service, the agent's long-running generation process should be handled asynchronously. When a user submits a request, the backend should immediately return a job ID and then process the video in the background. The frontend can poll an endpoint for status updates or receive notifications via WebSockets or webhooks.
- **Model and Asset Storage:** In a deployed environment, hard-coding file paths is not feasible. Use a centralized storage solution like Amazon S3, Google Cloud Storage, or a self-hosted MinIO server. Models should be downloaded on-demand or pre-loaded into a shared volume accessible by all service containers.
- **Security and Authentication:** Secure all public-facing API endpoints. Implement user authentication for the web interface and use API keys to protect the backend services from unauthorized access.

By following this comprehensive guide, developers and researchers can construct a powerful and highly automated system for creating music videos, bridging the gap between high-level creative intent and low-level generative control.

References

1. Self-hosting Llama 3.1 70b or any 70b LLM affordably - Medium (https://abhinand05.medium.com/self-hosting-llama-3-1-70b-or-any-70b-llm-affordably-2bd323d72f8d)
2. Can You Run This LLM? VRAM Calculator (Nvidia GPU and Apple Silicon) - apxml.com (https://apxml.com/tools/vram-calculator)
3. GPU System Requirements for Qwen Models - apxml.com (https://apxml.com/posts/gpu-system-requirements-qwen-models)
4. Qwen2.5-72B - apxml.com (https://apxml.com/models/qwen2-5-72b)
5. Guide to GPU Requirements for Running AI Models - Bacloud (https://www.bacloud.com/en/blog/163/guide-to-gpu-requirements-for-running-ai-models.html)

6. Function Calling with Open-Source LLMs - BentoML (https://www.bentoml.com/blog/function-calling-with-open-source-llms)
7. Qwen2.5-VL-72B: VRAM, Performance, and Deployment Guide - Novita AI Blogs (https://blogs.novita.ai/qwen2-5-vl-72b-vram/)
8. LLM Serving: Ollama vs vLLM vs TGI | Guides | Clore.ai - Clore.ai (https://docs.clore.ai/guides/comparisons/llm-serving-comparison)
9. Multi-node & Multi-GPU inference with vLLM - MeluXina User Documentation - MeluXina User Documentation (https://docs.lxp.lu/howto/llama3-vllm/)
10. Distributed Inference and Serving — vLLM - vLLM (https://docs.vllm.ai/en/v0.8.0/serving/distributed_serving.html)
11. Parallelism and Scaling - vLLM - vLLM (https://docs.vllm.ai/en/stable/serving/parallelism_scaling/)
12. Deep Learning VRAM Calculator - GPU Memory Requirements for LLMs & AI | DataZier - datazier.com (https://www.datazier.com/free-tools/vram-calculator/)
13. ComfyUI-Workflow/awesome-comfyui - GitHub (https://github.com/ComfyUI-Workflow/awesome-comfyui)
14. Big-Idea-Technology/ComfyUI_LLM_Node - GitHub (https://github.com/Big-Idea-Technology/ComfyUI_LLM_Node)
15. gokayfem/ComfyUI_VLM_nodes - GitHub (https://github.com/gokayfem/ComfyUI_VLM_nodes)
16. kmlbdh/ComfyUI_LocalLLMNodes - GitHub (https://github.com/kmlbdh/ComfyUI_LocalLLMNodes)
17. lxe/ComfyUI-OpenAI-Compat-LLM-Node - GitHub (https://github.com/lxe/ComfyUI-OpenAI-Compat-LLM-Node)
18. pollockjj/ComfyUI-MultiGPU - GitHub (https://github.com/pollockjj/ComfyUI-MultiGPU)
19. vllm-project/vllm/issues/7228 - GitHub (https://github.com/vllm-project/vllm/issues/7228)
20. XiongjieDai/GPU-Benchmarks-on-LLM-Inference - GitHub (https://github.com/XiongjieDai/GPU-Benchmarks-on-LLM-Inference)
21. vLLM vs TGI vs TensorRT-LLM vs Ollama | Hivenet - Hivenet (https://compute.hivenet.com/post/vllm-vs-tgi-vs-tensorrt-llm-vs-ollama)
22. Multi-GPU LLM Serving: A Practical Guide | Hivenet - Hivenet (https://compute.hivenet.com/post/multi-gpu-llm-serving)
23. vLLM Performance Benchmarks (4x RTX 3090) - himeshp.blogspot.com (http://himeshp.blogspot.com/2025/03/vllm-performance-benchmarks-4x-rtx-3090.html)
24. Discussion on Llama-3.1-Nemotron-70B-Instruct-HF - Hugging Face (https://huggingface.co/nvidia/Llama-3.1-Nemotron-70B-Instruct-HF/discussions/22)
25. Discussion on Qwen2-72B - Hugging Face (https://huggingface.co/Qwen/Qwen2-72B/discussions/3)
26. Discussion on Qwen2.5-VL-72B-Instruct - Hugging Face (https://huggingface.co/Qwen/Qwen2.5-VL-72B-Instruct/discussions/3)
27. Best VRAM Cheat Sheet for Local LLMs: Every Model, Every Quant | InsiderLLM - insiderllm.com (https://insiderllm.com/guides/vram-requirements-local-llms/)
28. vLLM vs Ollama vs llama.cpp vs TGI vs TensorRT-LLM: 2025 Guide - ITECS Team (https://itecsonline.com/post/vllm-vs-ollama-vs-llama.cpp-vs-tgi-vs-tensort)
29. Benchmarking NVIDIA TensorRT-LLM - Jan.ai (https://www.jan.ai/post/benchmarking-nvidia-tensorrt-llm)
30. Best GPUs for LLM Inference in 2025 - localllm.in (https://localllm.in/blog/best-gpus-llm-inference-2025)
31. Ollama VRAM Requirements for Local LLMs - localllm.in (https://localllm.in/blog/ollama-vram-requirements-for-local-llms)

32. Mastering LLM Tool Calling: The Complete Framework for Connecting Models to the Real World - MachineLearningMastery.com (https://machinelearningmastery.com/mastering-llm-tool-calling-the-complete-framework-for-connecting-models-to-the-real-world/)
33. GPU Inference Servers Comparison: Triton vs TGI vs vLLM vs Ollama - Nir Adler's Blog (https://blog.niradler.com/gpu-inference-servers-comparison-triton-vs-tgi-vs-vllm-vs-ollama)
34. Function Calling with LLMs | Prompt Engineering Guide - Prompt Engineering Guide (https://www.promptingguide.ai/applications/function_calling)
35. Function Calling - Qwen - Qwen Docs (https://qwen.readthedocs.io/en/latest/framework/function_call.html)
36. r/AI_Agents on Reddit: Function Calling in LLMs – Real Use Cases and Value? - Reddit (https://www.reddit.com/r/AI_Agents/comments/1iio39z/function_calling_in_llms_real_use_cases_and_value/)
37. r/LocalLLaMA on Reddit: Llama3-70B performs very similarly to Mistral - Reddit (https://www.reddit.com/r/LocalLLaMA/comments/1c89hj3/llama370b_performs_very_similarly_to_mistral/)
38. r/LocalLLaMA on Reddit: New RAG benchmark including Llama3 70B and 8B - Reddit (https://www.reddit.com/r/LocalLLaMA/comments/1c9whsv/new_rag_benchmark_including_llama3_70b_and_8b/)
39. r/LocalLLaMA on Reddit: Llama 3 70B Instruct works surprisingly well on... - Reddit (https://www.reddit.com/r/LocalLLaMA/comments/1cj4det/llama_3_70b_instruct_works_surprisingly_well_on/)
40. r/LocalLLaMA on Reddit: Are there any good newer models similar to... - Reddit (https://www.reddit.com/r/LocalLLaMA/comments/1ecnupa/are_there_any_good_newer_models_similar_to/)
41. r/LocalLLaMA on Reddit: Hardware requirements to run Llama 3 70B on a... - Reddit (https://www.reddit.com/r/LocalLLaMA/comments/1eiwnqe/hardware_requirements_to_run_llama_3_70b_on_a/)
42. r/comfyui on Reddit: Please explain the use of VRAM and some models... - Reddit (https://www.reddit.com/r/comfyui/comments/1nk6pso/please_explain_the_use_of_vram_and_some_models/)
43. r/StableDiffusion on Reddit: Manually setting VRAM limits in Comfy for... - Reddit (https://www.reddit.com/r/StableDiffusion/comments/1ewq140/manually_setting_vram_limits_in_comfy_for/)
44. r/StableDiffusion on Reddit: Are dual GPUs out of the question for local AI... - Reddit (https://www.reddit.com/r/StableDiffusion/comments/1i8y03g/are_dual_gpus_out_of_the_question_for_local_ai/)
45. ComfyUI-MultiGPU - ComfyUI Nodes - RunComfy (https://www.runcomfy.com/comfyui-nodes/ComfyUI-MultiGPU)
46. Benchmarking Local Models: Minimax25 vs Llama 3 vs Mistral - SitePoint (https://www.sitepoint.com/benchmarking-local-models-minimax25-vs-llama-3-vs-mistral/)
47. Local LLM Hardware Requirements: Mac vs PC 2026 | SitePoint - SitePoint (https://www.sitepoint.com/local-llm-hardware-requirements-mac-vs-pc-2026/)
48. Ollama vs vLLM: When to Scale Your Local AI Stack - SitePoint (https://www.sitepoint.com/ollama-vs-vllm-scaling-local-ai-stack)
49. Nvidia's RTX 3090 is a surprisingly capable AI workhorse - The Register (https://www.theregister.com/2024/08/23/3090_ai_benchmark/)
50. Running private LARGE AI Models on Your Hardware: A Guide to Scaling Local LLMs - Weisser Zwerg (https://weisser-zwerg.dev/posts/local-llm-large-llms/)
51. https://comfy.icu/docs/api (https://comfy.icu/docs/api)
52. https://github.com/SaladTechnologies/comfyui-api (https://github.com/SaladTechnologies/comfyui-api)

53. https://comfyai.run/documentation/Rest%20Api%20Node (https://comfyai.run/documentation/Rest%20Api%20Node)
54. https://9elements.com/blog/hosting-a-comfyui-workflow-via-api/ (https://9elements.com/blog/hosting-a-comfyui-workflow-via-api/)
55. https://docs.comfy.org/specs/workflow_json (https://docs.comfy.org/specs/workflow_json)
56. https://medium.com/@alwayswithya/getting-started-with-comfyui-from-json-generation-requests-435380ced924 (https://medium.com/@alwayswithya/getting-started-with-comfyui-from-json-generation-requests-435380ced924)
57. https://comfyui-wiki.com/en/interface/workflow (https://comfyui-wiki.com/en/interface/workflow)
58. https://github.com/aimpowerment/comfyui-workflows (https://github.com/aimpowerment/comfyui-workflows)
59. https://www.reddit.com/r/comfyui/comments/1l1gema/share_your_best_workflow_json_models/ (https://www.reddit.com/r/comfyui/comments/1l1gema/share_your_best_workflow_json_models/)
60. https://www.reddit.com/r/comfyui/comments/1952k8p/is_it_possible_to_run_comfyui_workflows_entirely/ (https://www.reddit.com/r/comfyui/comments/1952k8p/is_it_possible_to_run_comfyui_workflows_entirely/)
61. https://github.com/pydn/ComfyUI-to-Python-Extension (https://github.com/pydn/ComfyUI-to-Python-Extension)
62. https://www.timlrx.com/blog/executing-comfyui-workflows-as-standalone-scripts/ (https://www.timlrx.com/blog/executing-comfyui-workflows-as-standalone-scripts/)
63. https://www.reddit.com/r/StableDiffusion/comments/15muvti/automate_your_comfyui_workflows_with_the_comfyui/ (https://www.reddit.com/r/StableDiffusion/comments/15muvti/automate_your_comfyui_workflows_with_the_comfyui/)
64. https://www.reddit.com/r/StableDiffusion/comments/187b4dm/how_do_i_batch_process_images_on_comfyui/ (https://www.reddit.com/r/StableDiffusion/comments/187b4dm/how_do_i_batch_process_images_on_comfyui/)
65. https://github.com/Zar4X/ComfyUI-Batch-Process (https://github.com/Zar4X/ComfyUI-Batch-Process)
66. https://www.reddit.com/r/comfyui/comments/15vkhl1/batch_processing/ (https://www.reddit.com/r/comfyui/comments/15vkhl1/batch_processing/)
67. https://www.comflowy.com/blog/comflowy-batch (https://www.comflowy.com/blog/comflowy-batch)
68. https://comfy.icu/extension/Zar4X__ComfyUI-Batch-Process (https://comfy.icu/extension/Zar4X__ComfyUI-Batch-Process)
69. https://github.com/AIDC-AI/ComfyUI-Copilot (https://github.com/AIDC-AI/ComfyUI-Copilot)
70. https://developers.masterpiecex.com/comfyui-nodes (https://developers.masterpiecex.com/comfyui-nodes)
71. https://comfyui.org/en/comfyui-copilot-ultimate-ui-workflow (https://comfyui.org/en/comfyui-copilot-ultimate-ui-workflow)
72. https://github.com/comfyanonymous/ComfyUI (https://github.com/comfyanonymous/ComfyUI)
73. https://docs.comfy.org/registry/api-reference/overview (https://docs.comfy.org/registry/api-reference/overview)
74. https://medium.com/@next.trail.tech/how-to-use-comfyui-api-with-python-a-complete-guide-f786da157d37 (https://medium.com/@next.trail.tech/how-to-use-comfyui-api-with-python-a-complete-guide-f786da157d37)
75. https://github.com/emjjkk/beat-detection (https://github.com/emjjkk/beat-detection)
76. https://www.opus.pro/blog/best-ai-beat-sync (https://www.opus.pro/blog/best-ai-beat-sync)

77. ComfyUI ACE-Step Native Example - ComfyUI (https://docs.comfy.org/tutorials/audio/ace-step/ace-step-v1)
78. ComfyUI | Generate video, images, 3D, audio with AI (https://www.comfy.org/)
79. Free Video: Creating AI Music Videos with ComfyUI - Local Installation Guide from Vladimir Chopine [GeekatPlay] | Class Central (https://www.classcentral.com/course/youtube-how-to-create-a-stunning-ai-music-video-with-local-installation-430467)
80. https://dl.acm.org/doi/fullHtml/10.1145/3478384.3478420 (https://dl.acm.org/doi/fullHtml/10.1145/3478384.3478420)
81. https://www.yeschat.ai/gpts-9t557WIFeVi-Music-Lyric-Art-Generator (https://www.yeschat.ai/gpts-9t557WIFeVi-Music-Lyric-Art-Generator)
82. Music Video Generator | ComfyUI Workflow | OpenArt (https://openart.ai/workflows/fox_young_48/music-video-generator/OFG1thsX8vbjRRt7ITIg)
83. https://mwm.ai/apps/bpm-detector-tempo-analyzer/6759620195 (https://mwm.ai/apps/bpm-detector-tempo-analyzer/6759620195)
84. https://bpm-finder.net (https://bpm-finder.net)
85. https://www.neuralframes.com/lyric-video-maker (https://www.neuralframes.com/lyric-video-maker)
86. https://freebeat.ai/ (https://freebeat.ai/)
87. https://www.revid.ai/tools/ai-music-video-generator (https://www.revid.ai/tools/ai-music-video-generator)
88. https://info.invideo.io/task/blog/ai-creates-story-based-music-videos-from-lyrics (https://info.invideo.io/task/blog/ai-creates-story-based-music-videos-from-lyrics)
89. https://www.revid.ai/tools/suno-to-video (https://www.revid.ai/tools/suno-to-video)
90. https://www.yeschat.ai/gpts-ZxX7eSIS-Lyric-Visualizer (https://www.yeschat.ai/gpts-ZxX7eSIS-Lyric-Visualizer)
91. https://www.reddit.com/r/ofMontreal/comments/q3mh4m/lets_put_some_lyrics_into_that_ai_art_generator/ (https://www.reddit.com/r/ofMontreal/comments/q3mh4m/lets_put_some_lyrics_into_that_ai_art_generator/)
92. https://hookedoninnovation.com/2024/02/20/karaioke-when-ai-art-meets-music/ (https://hookedoninnovation.com/2024/02/20/karaioke-when-ai-art-meets-music/)
93. r/comfyui on Reddit: Quick guide to making a Music Video with ComfyUI - Workflows included (https://www.reddit.com/r/comfyui/comments/1k0qhed/quick_guide_to_making_a_music_video_with_comfyui/)
94. ComfyUI Workflows | Runnable Guaranteed with Pre-set Nodes & Models (https://www.runcomfy.com/comfyui-workflows)
95. https://oxelon.com/t/beat-detection (https://oxelon.com/t/beat-detection)
96. 🎵 Generate Music in Minutes: Introducing ComfyUI_DiffRhythm v1.0.0! - ComfyUI.org (https://comfyui.org/en/generate-music-with-comfyui-diffrhythm)